

INSTITUTO TECNOLÓGICO DE COSTA RICA

ESCUELA DE INGENIERÍA EN COMPUTADORES

CURSO: CE-1104 FUNDAMENTOS DE SISTEMAS

COMPUTACIONALES

DOCUMENTO TÉCNICO

PROYECTO: STRANGERTEC MORSE TRANSLATOR:

INCREMENTADOR EN 5

PROFESOR:

SANTIAGO GAMBOA RAMÍREZ

ESTUDIANTES:

ERNESTO MORA MARTÍNEZ

SANTIAGO PORTUGUEZ HERNÁNDEZ

FECHA: 24 JUNIO 2026

INTRODUCCIÓN

El presente documento técnico describe la extensión desarrollada para el Proyecto II del juego StrangerTEC Morse Translator, derivado del Proyecto I. Esta etapa agrega un circuito lógico combinatorio, construido físicamente con compuertas lógicas, capaz de incrementar en 5 el valor representado por los 4 bits menos significativos del código ASCII de una letra detectada mediante código Morse.

Durante la partida, cuando el switch de habilitación está activo y un jugador ingresa una letra en código Morse, el sistema detecta la letra, obtiene su código ASCII, lo convierte a binario y extrae sus 4 bits menos significativos. Esos bits alimentan las entradas del circuito incrementador en 5, cuya salida se visualiza en 4 LEDs adicionales de la maqueta y, de forma simultánea, se muestra en la interfaz del juego para comparar el resultado físico con el calculado por software.

La comunicación entre la computadora y la Raspberry Pi Pico W se realiza de forma inalámbrica mediante WiFi, usando sockets TCP. La interfaz de la computadora está desarrollada en Python con la biblioteca Pygame, y el firmware de la Pico W en MicroPython. Este documento presenta el diseño lógico del circuito (tabla de verdad, simplificación y diagrama), la arquitectura del sistema, los módulos de software y el análisis de los resultados obtenidos.

CONCLUSIONES

El proyecto permitió ampliar el conocimiento obtenido del proyecto 1, ya que se partió de un juego de código Morse que se le incorporó una etapa combinatoria completamente nueva, construida físicamente en hardware. Gracias a esta integración, una acción tan sencilla como ingresar una letra en Morse durante la partida se convierte en una señal eléctrica observable: el sistema toma el código ASCII de la letra, extrae sus cuatro bits menos significativos y los entrega a un circuito que incrementa ese valor en cinco. Un punto importante por destacar es que dicho circuito se implementó utilizando exclusivamente compuertas lógicas de la familia 74HC (74HC04, 74HC08, 74HC32 y 74HC86), sin recurrir a sumadores integrados ni a soluciones prefabricadas. Cumpliendo con el requisito de diseñar la lógica a partir de compuertas individuales y aprendiendo el comportamiento de cada función booleana en el diseño.

En cuanto a los resultados, la verificación del circuito fue exhaustiva: se recorrieron las dieciséis combinaciones posibles de entrada y, en todos los casos, las salidas físicas S3, S2, S1 y S0 coincidieron tanto con la tabla de verdad como con el valor calculado por el software. Consideramos que un factor decisivo para alcanzar este nivel de confiabilidad fue la organización del código en módulos bien diferenciados como la captura del Morse, lógica del juego, comunicación y control del circuito, ya que esa separación facilitó enormemente la etapa de depuración. Cuando algo no funcionaba, en lugar de revisar todo el sistema a la vez, fue posible aislar el problema y probar una sola señal a la vez, lo que permitió localizar de manera sistemática varios errores de hardware, como conexiones faltantes entre integrados o un riel de alimentación interrumpido.

Finalmente, el proyecto incorporó dos elementos que aportan robustez y realismo al funcionamiento del sistema. Por un lado, se añadió un switch de habilitación que permite activar o desactivar el circuito a voluntad; su estado se lee mediante un pin de la Raspberry Pi Pico y se reporta a la interfaz junto con el resultado, de modo que el usuario siempre sabe si la etapa combinatoria está operando. Por otro lado, y quizás como el cambio más relevante respecto a versiones anteriores, la comunicación entre la computadora y la maqueta se realizó de forma inalámbrica mediante WiFi, empleando un socket TCP en lugar del cable USB. Permitiendo que ambos dispositivos intercambiaran comandos, frases y resultados en tiempo real durante una partida, sin depender de una conexión física. En conjunto, todas estas características confirman que los objetivos se cumplieron de manera satisfactoria y que el sistema final es funcional, verificable y coherente.

RECOMENDACIONES

Una de las recomendaciones más importantes para quien quiera replicar o continuar este proyecto tiene que ver con el manejo correcto de la alimentación de los integrados, ya que de ello depende que el circuito funcione y que los componentes no se dañen. Es fundamental alimentar los integrados de la familia 74HC a 3.3 V tomados directamente de la Raspberry Pi Pico W, y nunca a 5 V, debido a que los pines de la Pico no son tolerantes a ese voltaje y podrían sufrir daños permanentes. Asimismo, conviene verificar siempre que cada integrado reciba su alimentación de forma adecuada, conectando el pin 14 a 3.3 V y el pin 7 a GND, y comprobar que el riel de alimentación de la protoboard no esté partido a la mitad, un detalle que pasa fácilmente desapercibido pero que puede dejar sin energía a parte del circuito. Estas verificaciones, aunque parezcan básicas, evitan muchos problemas difíciles de diagnosticar más adelante.

En cuanto al cableado y la lógica del circuito, recomendamos prestar atención al tratamiento de las entradas y salidas de cada compuerta. Las entradas que no se utilicen deben conectarse a GND para evitar lecturas inestables o flotantes que generen resultados erráticos; sin embargo, jamás se debe conectar una salida directamente a tierra, ya que esto provoca un cortocircuito que puede dañar el integrado o incluso la Pico. También resulta muy útil rotular claramente los LEDs de salida como S3, S2, S1 y S0, dejando indicado que S3 corresponde al bit más significativo y S0 al menos significativo, pues esto facilita enormemente la lectura del resultado y reduce los errores de interpretación al momento de comparar lo que muestra el hardware con lo esperado. Un montaje ordenado y bien identificado ahorra mucho tiempo durante las pruebas.

Por último, en lo referente a las pruebas y a la comunicación entre los dispositivos, aconsejamos aprovechar al máximo las herramientas que el propio sistema ofrece. Antes de integrar el circuito con el juego, es recomendable utilizar el modo de prueba manual mediante el comando BITS: para recorrer las dieciséis combinaciones de entrada y confirmar que cada salida responda correctamente de forma aislada. Para la parte inalámbrica, hay que asegurarse de que la Pico W y la computadora estén conectadas a la misma red WiFi, y de que la dirección IP y el puerto configurado coincidan en ambos programas, ya que cualquier diferencia impide la comunicación. Finalmente, sugerimos comparar el resultado físico de los LEDs con el valor mostrado en pantalla en, al menos, los cinco casos que evalúa el rubro y, de ser posible, en las dieciséis combinaciones, lo cual brinda una verificación mucho más completa y confiable del funcionamiento del sistema.

ANÁLISIS DE RESULTADOS

El análisis se centra en el diseño lógico del circuito incrementador en 5 y en la verificación de su funcionamiento. El sistema toma como entrada los 4 bits menos significativos del código ASCII de la letra detectada. En este documento se nombran A3, A2, A1 y A0, donde A3 es el bit más significativo y A0 el menos significativo, igual que en el firmware (GP2 = A0, GP3 = A1, GP4 = A2, GP5 = A3).

Tabla de verdad del incrementador en 5

La salida corresponde a sumar 5 unidades al valor de entrada, considerando únicamente 4 bits. Por esta razón, cuando el resultado supera 15, se conserva el resultado módulo 16.

Dec	A3	A2	A1	A0	Dec + 5 (mod 16)	S3	S2	S1	S0
0	0	0	0	0	5	0	1	0	1
1	0	0	0	1	6	0	1	1	0
2	0	0	1	0	7	0	1	1	1
3	0	0	1	1	8	1	0	0	0
4	0	1	0	0	9	1	0	0	1
5	0	1	0	1	10	1	0	1	0
6	0	1	1	0	11	1	0	1	1
7	0	1	1	1	12	1	1	0	0
8	1	0	0	0	13	1	1	0	1
9	1	0	0	1	14	1	1	1	0
10	1	0	1	0	15	1	1	1	1
11	1	0	1	1	0	0	0	0	0
12	1	1	0	0	1	0	0	0	1
13	1	1	0	1	2	0	0	1	0
14	1	1	1	0	3	0	0	1	1
15	1	1	1	1	4	0	1	0	0

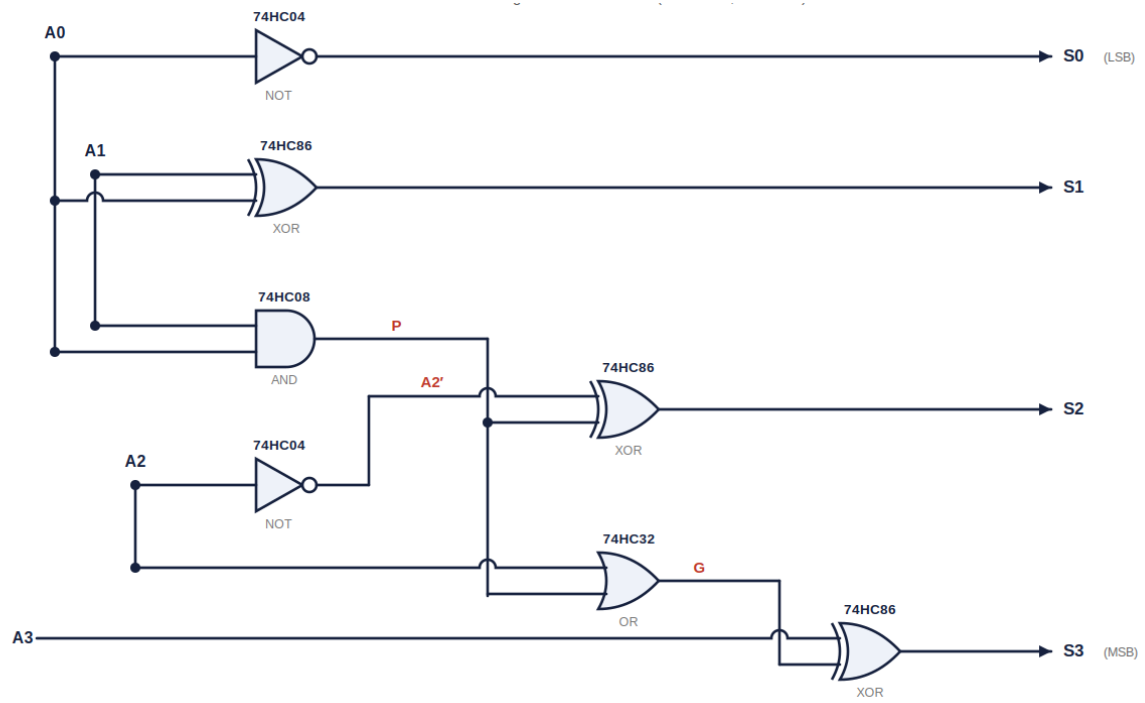
Simplificación del circuito

A partir de la suma del valor de entrada con la constante 0101 (5), considerando solo 4 bits, se obtienen las siguientes expresiones simplificadas. El término $P = A1 \cdot A0$ se calcula una sola vez y se reutiliza:

- $S0 = \text{NOT } A0$
- $S1 = A1 \text{ XOR } A0$
- $P = A1 \text{ AND } A0$
- $S2 = (\text{NOT } A2) \text{ XOR } P$
- $G = A2 \text{ OR } P$
- $S3 = A3 \text{ XOR } G$

Estas expresiones se implementan usando un integrado de cada tipo: el 74HC04 (NOT) genera NOT A0 y NOT A2; el 74HC08 (AND) genera P; el 74HC32 (OR) genera G; y el 74HC86 (XOR) genera S1, S2 y S3. No se utiliza una compuerta XNOR: el término (NOT A2) XOR P (equivalente a A2 XNOR P) se obtiene invirtiendo A2 con el 74HC04 y aplicando un XOR del 74HC86.

Diagrama del circuito



Materiales del circuito

Material	Uso dentro del Proyecto II
Raspberry Pi Pico W	Control de entradas, comunicación inalámbrica, lectura del switch y envío de los 4 bits hacia el circuito.
74HC04 (inversor / NOT)	Genera NOT A0 (salida S0) y NOT A2 (entrada del XOR de S2).
74HC08 (AND)	Genera el término $P = A1 \cdot A0$.
74HC32 (OR)	Genera el término $G = A2 + P$.
74HC86 (XOR)	Genera las salidas S1, S2 y S3.
Switch de habilitación	Activa o desactiva el circuito; su estado se lee por el pin GP6.
4 LEDs + resistencias (~330 Ω)	Visualización de S3, S2, S1 y S0, y limitación de corriente.
Protoboard y cables Dupont	Montaje físico y conexiones entre la Pico, las compuertas y los LEDs.

Verificación de resultados

Se verificó el circuito recorriendo las 16 combinaciones de entrada con el comando de prueba manual (BITS:) y comparando los LEDs con el valor calculado por software. La siguiente tabla resume las pruebas realizadas:

Prueba o actividad	Resultado obtenido	Estado
Verificación de entradas (modo prueba, BITS:)	Se ingresaron las 16 combinaciones desde el teclado de la computadora.	Correcto
Verificación de salidas en los LEDs	Las salidas S3, S2, S1 y S0 coinciden con la tabla de verdad.	Correcto
Bit de habilitación (switch GP6)	El estado del switch se lee y se reporta a la interfaz.	Correcto
Entrada desde código Morse (ASCII)	La letra detectada se convierte a ASCII y sus 4 bits menos significativos alimentan el circuito.	Correcto
Visualización del resultado en pantalla	El resultado del incrementador se muestra en la interfaz para compararlo con los LEDs.	Correcto
Comunicación inalámbrica (WiFi)	Intercambio de comandos y resultados entre la PC y la Pico W durante la partida.	Correcto

EXPLICACIÓN DE BIBLIOTECAS UTILIZADAS

Biblioteca / herramienta	Uso en el proyecto
Python 3	Lenguaje de la interfaz en la computadora: lógica del juego, conversión a ASCII y control general.
Pygame	Interfaz gráfica del juego: pantallas, eventos de teclado, visualización del Morse y del resultado del incrementador.
socket	Comunicación inalámbrica por TCP entre la computadora y la Pico W (se usa en ambos lados).
threading	Hilo en segundo plano que recibe los mensajes de la Pico sin bloquear la interfaz (lado PC).
diffliib (SequenceMatcher)	Cálculo de la similitud entre la frase objetivo y la respuesta del jugador para asignar puntajes.
dataclasses / typing	Estructuras de datos para los resultados de frases y rondas.
MicroPython	Lenguaje del firmware de la Raspberry Pi Pico W.
network (MicroPython)	Conexión de la Pico W a la red WiFi en modo estación.
machine (MicroPython)	Manejo de pines GPIO: lectura del botón y del switch, escritura de los 4 bits y control del LED y el buzzer (Pin, PWM).
time / uselect (MicroPython)	Medición de tiempos (puntos y rayas) y lectura no bloqueante del socket.

DIAGRAMA DE ARQUITECTURA BÁSICO

El sistema se compone de la interfaz en la computadora (Pygame), el firmware de la Raspberry Pi Pico W (MicroPython) y la maqueta con el circuito combinatorio. La computadora y la Pico W se comunican de forma inalámbrica por WiFi mediante un socket TCP, intercambiando comandos, el código Morse en vivo y el resultado del incrementador.

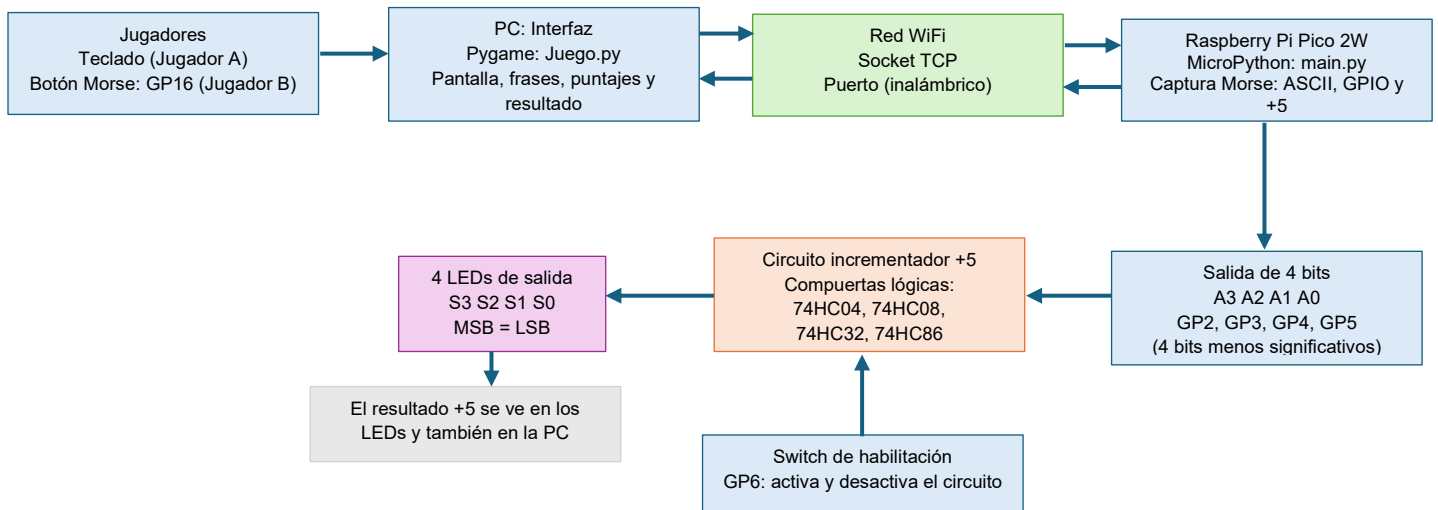
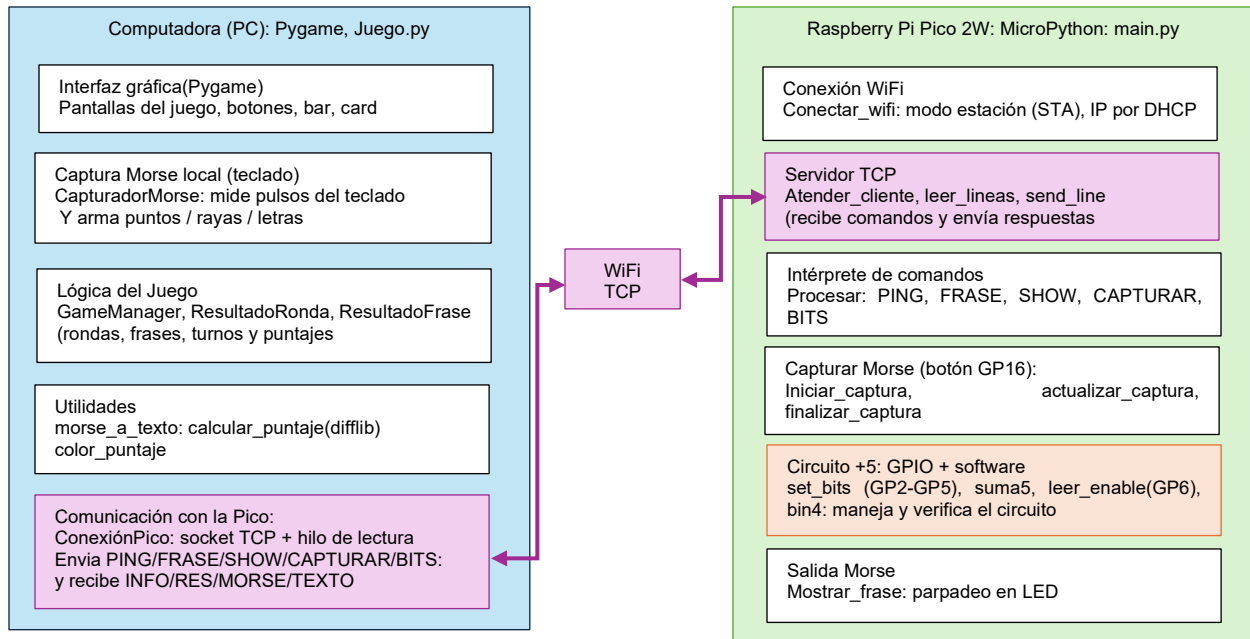


DIAGRAMA DE MÓDULOS DEL SISTEMA

El software se organiza en dos programas: Juego.py, que corre en la computadora con Pygame, y main.py, que corre en la Raspberry Pi Pico W con MicroPython. Ambos se comunican por WiFi. La siguiente figura muestra los módulos de cada lado y su relación con el hardware.



Funcionalidad del Proyecto

Módulo / archivo	Responsabilidad
Juego.py (computadora)	Interfaz Pygame, captura del Morse por teclado, lógica del juego (rondas, frases y puntajes), comunicación por socket y visualización del resultado.
ConexionPico (clase en Juego.py)	Administra el socket TCP y un hilo de lectura; envía los comandos y recibe las respuestas de la Pico.
main.py (Raspberry Pi Pico W)	Conexión WiFi, servidor TCP, intérprete de comandos, captura del Morse por botón, control del circuito +5 (GPIO) y salida en LED y buzzer.
Circuito +5 (funciones en main.py)	set_bits, suma5 y leer_enable: escriben los 4 bits hacia el circuito y comparan con el resultado calculado por software.
Maqueta (hardware)	Circuito combinacional con 74HC04, 74HC08, 74HC32 y 74HC86, 4 LEDs (S3..S0) y switch de habilitación.

BIBLIOGRAFÍA

International Telecommunication Union. (2009). ITU-R M.1677-1: International Morse code.
<https://www.itu.int/rec/R-REC-M.1677-1-200910-I/en>

MicroPython. (2024). MicroPython documentation. <https://micropython.org/>

Pygame Community. (2024). Pygame documentation. <https://www.pygame.org/docs/>

Python Software Foundation. (2024). Python documentation. <https://docs.python.org/3/>

Raspberry Pi Foundation. (2024). Raspberry Pi Pico W documentation.
<https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html>

Raspberry Pi Ltd. (2024). *RP2040 datasheet: A microcontroller by Raspberry Pi* [Hoja de datos]. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>

Texas Instruments. (s. f.). *SNx4HC04 hex inverters* [Hoja de datos].
<https://www.ti.com/lit/ds/symlink/sn74hc04.pdf>

Texas Instruments. (s. f.). *SNx4HC08 quadruple 2-input AND gates* [Hoja de datos].
<https://www.ti.com/lit/ds/symlink/sn74hc08.pdf>

Texas Instruments. (s. f.). *SNx4HC32 quadruple 2-input OR gates* [Hoja de datos].
<https://www.ti.com/lit/ds/symlink/sn74hc32.pdf>

Texas Instruments. (s. f.). *SNx4HC86 quadruple 2-input exclusive-OR gates* [Hoja de datos]. <https://www.ti.com/lit/ds/symlink/sn74hc86.pdf>

Sweigart, A. (2012). *Making games with Python & Pygame*.
<https://inventwithpython.com/pygame/>

Anthropic. (2025). Claude (claude-sonnet-4-20250514) [Modelo de lenguaje].
<https://www.anthropic.com>